КНИГА 2

Промпт-инжиниринг

Как формулировать запросы к ИИ, чтобы получать нужный результат

Уровень: Начинающий — Продвинутый
Библиотека ИИ · 5 книг · все уровни и категории

1. Почему формулировка имеет значение

Начинающий

Языковая модель не «понимает» намерение так, как это делает человек — она статистически продолжает текст на основе того, что вы написали. Это значит, что два запроса с одинаковым смыслом, но разной формулировкой, могут дать заметно разные по качеству ответы. Промпт-инжиниринг — это навык формулировать запросы так, чтобы модель с наибольшей вероятностью поняла именно то, что вы имели в виду.

2. Пять базовых принципов

Начинающий

Будьте конкретны

Вместо «напиши про маркетинг» — «напиши 5 заголовков email-рассылки для запуска нового кроссовка, тон — энергичный, до 60 символов каждый».

Задавайте формат ответа

Если нужен список, таблица или JSON — скажите об этом явно, а не надейтесь, что модель угадает.

Давайте контекст

Чем больше релевантного контекста (аудитория, цель, ограничения), тем точнее результат.

Разбивайте сложное на шаги

Длинную многоэтапную задачу лучше описать по шагам, а не одной размытой фразой.

Показывайте примеры

Один-два примера желаемого результата (few-shot) часто работают лучше длинных объяснений.

3. Zero-shot, one-shot и few-shot промптинг

Начинающий

Это способы показать модели, что вы хотите, через количество примеров прямо в запросе.

Zero-shot (без примеров)

```
Классифицируй тональность отзыва как позитивную, негативную или нейтральную:
"Доставка задержалась на неделю, но сам товар отличный."
```

Few-shot (с примерами)

```
Отзыв: "Товар сломался на второй день" -> Негативная
Отзыв: "Всё супер, спасибо!" -> Позитивная
Отзыв: "Доставка задержалась на неделю, но сам товар отличный." -> ?
```

Few-shot особенно полезен, когда нужен специфический формат ответа или нестандартная классификация, которую трудно описать словами, но легко показать примером.

4. Chain-of-Thought (цепочка рассуждений)

Средний

Для задач, требующих логики или нескольких шагов вычислений, качество ответа заметно растёт, если явно попросить модель рассуждать по шагам перед финальным ответом, вместо того чтобы сразу выдавать итог.

```
В корзине было 23 яблока. Съели 7, затем докупили 10.
Сколько яблок в корзине сейчас?
Решай по шагам, прежде чем дать окончательный ответ.
```

Примечание. Для современных «рассуждающих» моделей (reasoning models) это зачастую происходит автоматически внутри модели, но для обычных моделей явная просьба «подумай по шагам» всё ещё заметно повышает точность.

5. Системные промпты и роли

Средний

Многие интерфейсы и API позволяют задать «системный промпт» — инструкцию, которая задаёт поведение модели на весь разговор, в отличие от обычных сообщений пользователя.

```
system: "Ты — строгий технический рецензент кода на Python. Отвечай кратко, указывай
номера строк, не используй вежливые вступления."
```

Роль (персона) полезна, когда нужен устойчивый стиль или фокус на протяжении всего диалога: «отвечай как юрист», «отвечай как редактор», «отвечай только фактами без мнений».

6. Структурирование сложных запросов

Средний

Для сложных задач полезно явно разделять части запроса — например, с помощью заголовков или XML-подобных тегов — чтобы модель не путала, где контекст, где данные, а где инструкция.

```
<контекст>
Ты помогаешь составить резюме встречи.
</контекст>

<транскрипт>
...текст встречи...
</транскрипт>

<задача>
Выдели 3 ключевых решения и 3 открытых вопроса.
</задача>
```

7. Структурированный вывод (JSON) для разработчиков

Продвинутый

При интеграции модели в программный код часто нужен предсказуемый машиночитаемый формат, а не свободный текст.

```
Извлеки из текста ниже имя, дату и сумму.
Верни ТОЛЬКО валидный JSON по схеме:
```

```
{
  "name": string,
  "date": string (YYYY-MM-DD),
  "amount": number
}

Текст: "Иван оплатил 450 леев 3 марта."
```

На практике многие API (включая Claude и аналоги) поддерживают нативные режимы структурированного вывода или «инструментов» (tool use), что надёжнее, чем просить JSON одним текстом.

8. Итеративный промптинг

Начинающий

Редко с первого раза получается идеальный результат — и это нормально. Эффективный подход: получить черновик, явно указать, что изменить («короче», «более формально», «добавь пример»), и повторить. Диалог с моделью — это итеративный процесс уточнения, а не одна попытка угадать идеальный промпт.

9. Частые ошибки новичков

Начинающий

- Слишком общий запрос без контекста о цели и аудитории
- Просьба о количестве («напиши 10 идей») без критериев качества
- Ожидание, что модель помнит прошлые отдельные сессии без явного контекста
- Противоречивые инструкции в одном запросе («будь кратким» + «опиши всё подробно»)
- Доверие цифрам и фактам без проверки в ответственных задачах

10. Продвинутые техники для разработчиков

Продвинутый

Самосогласованность (self-consistency)

Запуск одного и того же промпта несколько раз с рассуждениями и выбор наиболее частого итогового ответа — повышает надёжность на задачах с чёткими правильными ответами.

ReAct (Reasoning + Acting)

Паттерн, в котором модель чередует шаги рассуждения с вызовом внешних инструментов (поиск, калькулятор, код) и наблюдением за результатом, прежде чем дать финальный ответ — основа большинства современных агентных систем.

Цепочки промптов (prompt chaining)

Разбиение сложной задачи на последовательность более простых промптов, где выход одного шага становится входом следующего — надёжнее, чем один гигантский промпт на всё сразу.

Негативные инструкции (negative prompting)

Явное указание, чего делать НЕ нужно («не используй жаргон», «не придумывай источники, которых нет»), работает, но менее надёжно, чем позитивная

формулировка желаемого поведения — по возможности формулируйте требования в положительном ключе.

11. Итоговый чек-лист

Начинающий

- ☐ Указана конкретная цель и аудитория
- ☐ Указан желаемый формат ответа
- ☐ Дан необходимый контекст (но не избыточный)
- ☐ Сложная задача разбита на шаги
- ☐ При необходимости приведён пример желаемого результата
- ☐ Проверены факты и цифры в ответственных задачах